

Relatório — TP1: pipeline de visão em ROS 2

PB Sistemas Robóticos 2026.2 · turma GRPEDCR3C1-M1-P1 · entrega entrega-tp1 / tag tp1 · 28/08/2026

Aluno: Rodrigo das Mercês Vianna

GitHub: Dev-RMV (projeto-pb-Dev-RMV)

Ambiente: Ubuntu 22.04 (WSL2) · ROS 2 Humble
· Python 3.10 · OpenCV 4.5.4

Vídeo da demo: https://www.youtube.com/watch?v=WrpPA5194_8

1. O que foi pedido e o que foi entregue

Requisito	Entrega
Nó publisher em Python que capture frames de câmera ou vídeo com OpenCV e publique em <code>/camera/image_raw</code>	<code>camera_publisher</code> : <code>cv2.VideoCapture</code> (índice de câmera ou arquivo), <code>sensor_msgs/Image</code> via <code>cv_bridge</code> , taxa igual à da fonte
Nó subscriber que receba os frames e aplique conversão HSV e segmentação por cor	<code>color_segmenter</code> : <code>BGR→HSV</code> , <code>inRange</code> por faixa de cor (várias cores configuráveis), morfologia, contornos → objetos; publica máscara e frame anotado
Serviço <code>/vision/status</code> com o número de objetos segmentados no último frame	<code>/vision/status</code> (srv própria <code>VisionStatus</code>): total de objetos, contagem por cor e frames processados
Detecção de faces e extração de <i>features</i> nos frames recebidos	<code>face_features</code> : Haar cascade para rostos; por rosto, olhos (Haar), keypoints/descriptores ORB e vetor HOG; publica <code>/vision/faces</code> e <code>/vision/faces_image</code> ; serviço <code>/vision/face_status</code>

2. Como funciona

```
demo.mp4 → camera_publisher → /camera/image_raw
├→ color_segmenter → /vision/mask, /vision/segmented · srv /vision/status
└→ face_features → /vision/faces, /vision/faces_image · srv /vision/face_status
```

- **Entrada:** `demo.mp4` (768×432, 12 fps, 79 s, em loop) alterna blocos de 5 s entre uma bola azul quicando e cenas com pessoas — assim a segmentação e a detecção facial trabalham sobre os mesmos frames e os dois serviços mostram a alternância. `source:=0` troca o vídeo pela webcam.
- **Segmentação:** `GaussianBlur` → `HSV` → `inRange` para cada cor de `config/params.yaml` (azul, vermelho, verde, amarelo; o vermelho cruza `H = 0` e é tratado como duas faixas) → abertura/fechamento → `findContours`; contornos com área ≥ 1500 px contam como objeto. A máscara publicada é pintada na cor de cada faixa.
- **Rostos e features:** só OpenCV clássico, sem modelos baixados: `haarcascade_frontalface_alt2.xml` (escolhido por medição sobre o `_default`), olhos por `haarcascade_eye.xml` na metade superior do rosto, ORB com `fastThreshold 10` (pele tem pouca textura) e HOG do recorte 64×64 (1764 valores).
- **Interfaces próprias** (`projeto_bloco_interfaces`): `msg/Face`, `msg/FaceArray`, `srv/VisionStatus`, `srv/FaceStatus`.
- **Launch único** `projeto_bloco.launch.py` sobe os três nós com `params.yaml` e o perfil Fast-DDS.

3. Como executar

```
git clone https://github.com/Dev-RMV/projeto-pb-Dev-RMV.git && cd projeto-pb-Dev-RMV
./scripts/reproduzir.sh          # ou ./EXECUTAR.sh — instala o que faltar (setup.sh), compila e roda
a demo
```

`scripts/setup.sh` instala todas as dependências via `apt` (inclusive o ROS 2 Humble, se ausente). O script abre as janelas *segmentado*, *mascara* e *rostos* e imprime os dois serviços a cada 5 s; Ctrl+C encerra tudo. Testado num Ubuntu 22.04 recém-instalado: $\approx 1,5$ min do clone até os nós no ar.

4. Resultados

Medida	Resultado
Alternância observada em <code>/vision/status</code> × <code>/vision/face_status</code>	bloco bola: azul=1 ou 2, 0 rostos · bloco pessoas: 1–3 rostos, azul=1 apenas na camiseta azul do homem
Segmentação azul (948 frames)	bola: 1 objeto em 384 frames, 2 objetos em 92 (duas bolas), 0 em 4 (bola fora de cena); pessoas: 1 objeto em 144 frames (camiseta), 0 nos demais
Detecção de rostos (Haar)	≥ 1 rosto em 365 de 468 frames de pessoas (perdas = perfil/costas); pessoa frontal 126/144 frames; 1 falso positivo em 480 frames de bola
Features por rosto	2 olhos em $\approx 50\%$ dos rostos ($\approx 100\%$ nos frontais próximos); ORB ≈ 25 keypoints em média, 44–82 nos rostos próximos; HOG 1764 valores
Desempenho	segmentador processa 100 % dos frames (24/24 fps em 720p); <code>face_features</code> ≈ 7 ms/frame em 768×432 (Haar 4,8 + olhos 0,9 + ORB 0,5 + HOG 0,1)

5. Dificuldades e decisões

- **Fast-DDS descartava frames grandes:** o segmento de memória compartilhada padrão (512 KB) fazia frames 720p caírem para UDP fragmentado — chegavam 4–6 dos 24 fps. Resolvido com `config/fastdds_profile.xml` (segmento de 32 MB), aplicado pelo launch: 24/24 fps.
- **Encerramento limpo:** `rc1py.shutdown()` dentro de callback trava no Humble e o Ctrl+C sob `ros2 launch` chega duas vezes ao nó; tratados com exceção própria e SIGINT ignorado durante a limpeza.
- **Haar em vez de DNN:** YuNet/SFace detectam melhor e permitem reidentificação, mas exigem baixar modelos; a entrega usa somente cascades do sistema para ser reproduzível com `apt`. Reidentificação clássica (HOG/LBP) foi avaliada e descartada por não separar pessoas.
- **Calibração das cores:** faixas definidas por histograma de matiz do vídeo; a faixa "laranja" foi retirada do padrão porque pega pele e o piso ao sol.
- **Sem câmera no WSL2:** a demo usa vídeo; câmera real via `source:=0` (com `usbipd`).
- **Repositório:** vídeo versionado em `ros2_ws/midia/` (exceção ao `*.mp4` do `.gitignore`) e scripts marcados como executáveis no git, para que o clone rode direto.